实验二：搜索算法 实验报告

A* 最短路径 · Alpha-Beta 剪枝 · 竞争环境下的宝藏分配

邱浩峰 10245501462 · 当代人工智能实验课 · 2026 年 10 月 19 日

GitHub 仓库（全部代码与实验数据）：https://github.com/zechun115/qhf.git

1 实验结果总览

本实验实现了 A*（四种启发函数、三种平局策略、加权变体）、Minimax 与 Alpha-Beta 剪枝，并把 A* 与博弈搜索结合求解竞争环境下的宝藏分配问题。四道题在题面给定用例上全部与预期输出一致，由 `main.py` 自动比对（表 1）；在此基础上设计了 7 组对照实验（E1–E7），量化启发强度、平局策略、可采纳性边界、访问顺序与缓存策略的影响，所有数据由固定随机种子采集，可完整复现。

题目	算法	题面预期	本实现输出	一致
一 迷宫寻路	A*（曼哈顿）	8 点坐标序列	8 点坐标序列，代价 6	是
二 加权地形	A*（代价加权）	7	7	是
三 博弈树	Minimax / Alpha-Beta	5	5	是
四 宝藏 H=2	A* + Minimax / α-β	T2，−1	T2，−1	是
四 宝藏 H=4	A* + Minimax / α-β	T3，3	T3，3	是

表 1 四道题目输出与题面预期逐项比对

2 算法设计

A* 以格子坐标为状态，g(n) 为起点到 n 的实际代价，h(n) 为剩余代价下界，f = g + h 决定OPEN 堆的出队次序；同一状态允许重复入队，弹出时若已扩展则丢弃（惰性删除，免去 decrease-key）。四邻域移动，代价记为“进入目标格”的权值，因此 S 格不计代价、题目二的 E 格按题面记 0。

```
OPEN ← {(h(s), s)};  g[s] ← 0;  closed ← ∅
while OPEN ≠ ∅:
    n ← OPEN.pop_min()              # 键为 (f, tie-break)
    if n ∈ closed: continue         # 惰性删除旧条目
    closed ← closed ∪ {n};  expanded++
    if n = goal: 回溯 parent 输出路径; return
    for m ∈ neighbors(n):
        g' ← g[n] + cost(n→m);  if g' < g[m]: 更新 g,parent 并入堆
```

Alpha-Beta 在 Minimax 递归中维护窗口 (α, β)：MAX 节点某子节点回传值 v ≥ β 时，父 MIN 已存在不劣于 β 的选择，后续兄弟整枝剪去；MIN 节点 v ≤ α 时对称剪枝。剪枝只削减搜索开销，不改变根节点价值——本报告所有对照实验均以“根值不变”作为正确性校验。题目四把 A* 求得的关键点距离矩阵送入博弈层，状态为 (posA, posB, remaining, scoreA, scoreB, player)，叶子评价 U = ScoreA − ScoreB，A 最大化、B 最小化。

3 题目一：迷宫寻路与启发函数

题面 5×5 迷宫的 A* 返回 8 点序列 (0,0)→(0,1)→(0,2)→(1,2)→(1,3)→(1,4)→(2,4)→(3,4)，代价 6，与预期一致；起点被障碍封死时返回空列表。图 1 对比了 h=0 与曼哈顿两种启发：前者按“已走代价”层层扩散，在障碍周围形成波前；后者始终优先扩展 f 最小的格子，几乎沿一条准直线推进到终点，扩展数从 16 降到 14。

题目一：同一迷宫的两种启发函数搜索过程（格内数字为扩展次序）

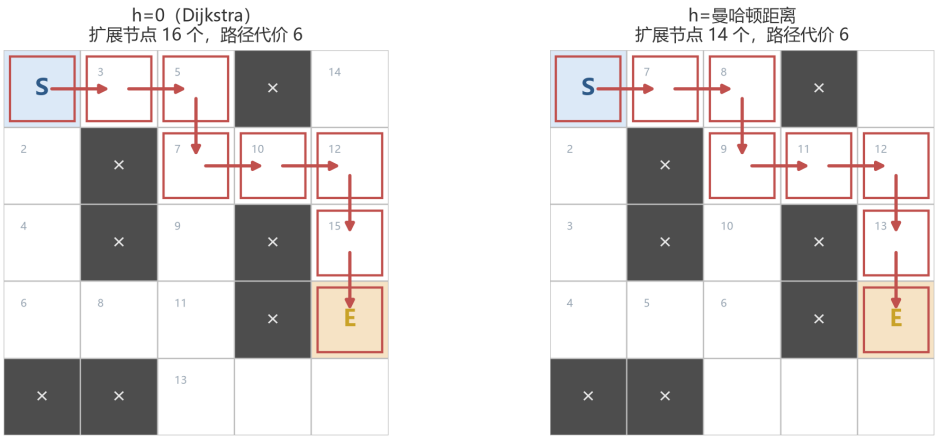


图 1 题面迷宫上的搜索过程：左 h=0（Dijkstra 波前），右曼哈顿启发；格内数字为扩展次序，红色为最终路径

启发函数	扩展节点	生成节点	OPEN 峰值	路径代价
h = 0（Dijkstra）	16	16	2	6
曼哈顿	14	16	3	6
切比雪夫	12	15	4	6
欧氏（取整）	12	15	4	6

表 2 题面 5×5 迷宫上四种启发函数的开销（最优代价均为 6）

地图规模	h = 0	曼哈顿	切比雪夫	欧氏	曼哈顿占比	h = 0 占比
21×21	307	83	221	215	18.9%	69.6%
41×41	1,174	279	907	882	16.6%	69.8%
61×61	2,638	430	2,144	2,105	11.6%	70.9%
81×81	4,642	981	3,933	3,881	15.0%	70.8%

表 3 随机迷宫（障碍率 28%，每档 20 张）：平均扩展节点数及占可通行格比例

分析。三种距离中曼哈顿是 4 邻域下最紧的下界：无障碍时剩余步数恰好等于曼哈顿距离，f 值的排序能力最强；切比雪夫与欧氏都只是曼哈顿的更弱下界，搜索行为介于 Dijkstra 与曼哈顿之间。规模放大了差距：81×81 上曼哈顿只探索约 15% 的可通行格，而 h=0 需 71%——后者本质上要淹没整片“代价 < 最优代价”的等高线区域才能确认最优。值得注意的是 5×5 上切比雪夫反而少扩展 2 个节点（12 对 14），说明小图上的单例对比不可靠，评估启发函数必须在大规模统计下进行。

E2 平局策略。同样是最优解，仅改变 f 值相同节点的出队次序，81×81 上平均扩展数就相差一倍：平局取 h 小 981 个，FIFO 1,924 个，平局取 h 大 2,000 个。h 小优先意味着在同等预算下优先推进离终点更近的节点，搜索带更窄；h 大优先则回头扩展浅层节点、接近广度优先。把次序键写成 (f, h) 是零成本优化。

E3 加权 A*。令 f = g + w·h 会破坏可采纳性，但能换速度：w=1.2 时扩展数从 981 降到 380（−61%），平均代价仅膨胀 2.4%；w=3 时扩展数趋于平缓（305），代价膨胀到 22.5%。启发权重是一个“次优度预算”旋钮：游戏寻路这类允许 2%~5% 次优的场合收益很大，严格最优场合必须保持 w=1。

4 题目二：加权地形与可采纳性边界

最优路径为 (0,0)→(1,0)→(2,0)→(3,0)→(3,1)→(3,2)→(4,2)→(4,3)→(4,4)，前 8 步每进入一个数字格累加其权值、末步进入 E 记 0，总代价 7，与预期一致。这条路线宁可绕行全为 1 的“平原”，也不穿中部的 8/3 高价地形——直线穿越的地形代价约为最优解的两倍。在“所有可通行格代价 ≥ 1、E 记 0”的前提下，曼哈顿仍然可采纳且一致：真实剩余代价 ≥ 剩余步数 ≥ 曼哈顿距离，且相邻格 h 差 ≤ 1 ≤ 最小步代价。

但可采纳不等于有效。权值 1–9 的 41×41 随机地图上，h=0 平均扩展 1,420 个节点，曼哈顿也只降到 1,413，几乎毫无帮助：真实代价约是步数的 5 倍，而 h 仍按每步 1 估计，启发项在 f 中占比过小，排序退化为 Dijkstra。启发函数的价值取决于它对真实代价结构的刻画能力，而不只是地图大小。自然的改进是用全图最小单步代价 c_min 缩放，h = c_min·曼哈顿 仍可采纳，且量级更贴近真实代价。

缩放系数 k	0（Dijkstra）	1	3	4.5	6
平均扩展节点	1,426	1,424	1,401	699	111
代价 / 最优	1.0000	1.0000	1.0000	1.0033	1.0672
返回次优解比例	0%	0%	0%	55%	100%

表 4 地形权值 3–9 的 41×41 随机地图（20 张）：h = k·曼哈顿 的边界扫描（c_min = 3）

E4 可采纳性边界。最小单步代价为 3 时，k=3 是理论上保持可采纳的上限，实验精确吻合：k ≤ 3 时 20 张地图全部返回最优解；k 越过下界后次优比例立即跳到 55%（k=4.5）与 100%（k=6），扩展数却从 1,401 骤降到 111（−92%）。可采纳性边界是一道“速度换正确性”的悬崖，越界一步，效率提升一个数量级而正确性立刻失守。这与 E3 的加权 A* 是同一个旋钮的两种写法。

E5 零代价格反例。若地图存在代价 0 的格子，上述论证全部失效：真实剩余代价可能小于剩余步数，曼哈顿开始高估。在 60 张含 35% 零代价格的 15×15 地图上，A* 在 28 张（46.7%）返回次优解，平均代价差 2.5，最大 8。最严重用例最优代价 28、A* 返回 36（差 8，相对误差 28.6%），而它扩展的节点数（83）反而少于求出最优解的 Dijkstra（152）——不可采纳的启发不仅牺牲最优性，还会制造“搜索很快”的假象，因为它系统性地把搜索引离真正廉价的区域。

5 题目三：Alpha-Beta 剪枝

题面树 Minimax 与 Alpha-Beta 均返回 5，与预期一致：Minimax 访问全部 15 个节点、评估 8 个叶子；Alpha-Beta 只访问 14 个节点、评估 7 个叶子。图 2 按真实执行顺序还原了数值回传与剪枝：① 根（MAX）处理完左子树后 α=5；② 左 MIN 节点取到 5 后 β=5，其第二个 MAX 子节点拿到窗口 (5, ∞)，评估叶子 6 得 6 ≥ β=5，父 MIN 已有更优的 5，叶子 9 被剪去（该 MAX 节点回传 6 只是真实值 9 的下界，父层取 min 时结论不变）；③ 右 MIN 回传 2 ≤ α=5，但它是最后一个孩子，已无兄弟可剪。

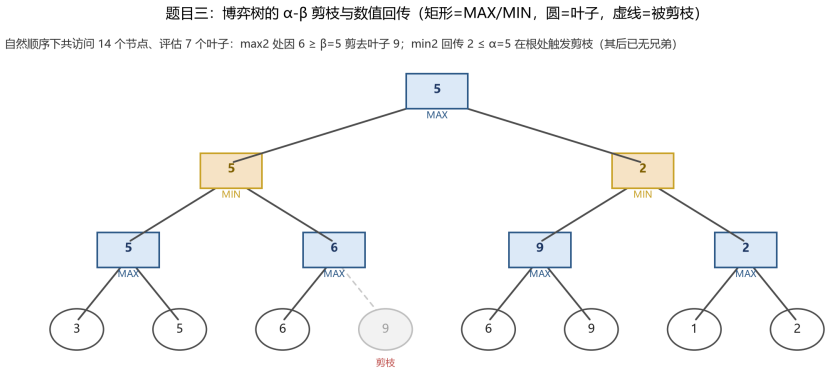


图 2 题面博弈树的 α-β 执行过程：矩形为 MAX/MIN 节点回传值，灰圆虚线为被剪枝而未评估的叶子（数值 9）

子节点顺序	自然顺序	最优优先	最差优先	随机
访问节点	14	11	15	11
评估叶子	7	5	8	5
根节点价值	5	5	5	5

表 5 题面树：四种子节点访问顺序的开销（根值恒为 5，验证剪枝不改变结果）

随机树（深度 6、分支 3，60 棵平均）	自然顺序	最优优先	最差优先	随机
平均访问节点	416	125	992	412
平均剪枝触发次数	96	46	150	94

表 6 E6 访问顺序对照（Minimax 平均 1,093 节点）

分析。第一，顺序决定效率：最优优先把节点数从 Minimax 的 1,093 压到 125（−88.6%），深度 7 时省 92.9%，接近完美排序下 O(b^(d/2)) 的理论极限（有效分支因子由 3.0 降到约 2.2）。第二，“最差优先”剪枝次数最多（149.5 次）却最慢（992 节点），因为它每次都在深层剪掉很小的子树；最优优先只触发 46 次，但每次都在浅层整枝跳过大子树——剪枝的价值在“剪在哪”，不在“剪几次”。第三，随机顺序（412）与自然顺序（416）几乎相同，说明不主动排序就享受不到 α-β 的理论收益；这也解释了实际博弈引擎为何把“走法排序”与搜索深度并列为两项核心技术。

6 题目四：竞争环境下的宝藏分配

对 A、B、T1–T4 六个关键点两两运行 A*（含障碍与加权地形），得到 6×6 最小移动代价矩阵（表 7）。矩阵沿对角线对称，交叉验证了 A* 实现；且两端点互跑代价相等，说明“进入格计代价、起点不计”的约定没有引入方向偏差。几个数字主导了整局：A 距 T2 只有 4、B 距 T3 只有 4，而 T4 远离一切（B→T4 为 10、T1→T4 达 12）却值 7 分——谁被迫最后去取 T4，谁就亏 5 分。

起点 \ 终点	A	B	T1	T2	T3	T4
A	0	12	10	4	8	8
B	12	0	8	8	4	10
T1	10	8	0	6	6	12
T2	4	8	6	0	6	6
T3	8	4	6	6	0	6
T4	8	10	12	6	6	0

表 7 A* 求得的 6×6 最小移动代价矩阵（对称）

用 Minimax 与 Alpha-Beta 分别在 H=2、4 求解，两者决策完全一致（再次验证剪枝不改决策）：H=2 时 A 的最优首步为 T2、U = −1；H=4 时为 T3、U = 3，均与题面预期一致。H=4 的最优对抗线为 A:T3（收益 −1）→ B:T1（+1）→ A:T2（0）→ B:T4（−5），U = (−1+0) − (1−5) = 3。

深度 H	Minimax 首步	α-β 首步	α-β 节点/剪枝（自然）	α-β 节点/剪枝（收益排序）
H = 2	T2（U=-1）	T2（U=-1）	13 / 2	11 / 3
H = 4	T3（U=3）	T3（U=3）	47 / 12	37 / 12

表 8 两种算法、两种子节点顺序在 H=2 / H=4 下的决策与开销

首步价值 U	T1	T2	T3	T4
H = 1	-1	+2	-1	-1
H = 2	-4	-1	-2	-4
H = 3	-4	+2	-1	-4
H = 4	-5	+1	+3	-7

表 9 各首步的博弈值（正=A 优；灰底为各深度下的最优首步）

为什么浅层与深层给出不同答案？H=1 时纯贪心：T2 收益 6−4=2 最高；H=2 时 A 取 T2 后 B 就地取 T3 得 3，U=2−3=−1，但其余首步 U ≤ −2，T2 仍是最不差的选择；到了 H=4，T3 以 −1 的“弃子”开局，却换来两重好处——A 次手就近收获 T2（收益 0），B 的次手被逼到 T1→T4，进入代价 12、收益 −5，U 升到 3，比 T2 首步（U=1）高 2 分。深度奇偶性还提示：H 为偶数时最后一手是对手，评价里包含“对手被迫收拾残局”的效应，因此 U 不能跨深度比较（H=3 时 T2 首步 U=2），但同一深度内的排序可靠——实际系统应让搜索深度与回合结构匹配，偶数深度才能看到对手的完整应对。

E7 距离缓存。H=4 的博弈搜索要计算 80 次收益；由于位置只在 6 个关键点间变化，把 24 对点对预计算成矩阵后，完整求解从 5.6 ms 降到 0.1 ms（约 56 倍），按需缓存居中（1.3 ms）。再叠加收益降序的子节点排序，α-β 节点数由 47 降到 37（−21%）。这说明把不变量从搜索循环中提出来，往往比换算法更划算。

7 对照实验汇总与结论

实验	设置	关键结论
E1 启发强度	曼哈顿 / 切比雪夫 / 欧氏 / h=0	81×81 上曼哈顿探索 15% 可通行格，h=0 需 71%
E2 平局策略	(f,h) / (f,−h) / FIFO	h 小优先比 h 大优先少扩展一半（981 对 2,000）
E3 加权 A*	w = 1.0 → 3.0	w=1.2：节点 −61%，代价仅 +2.4%；w=3 时代价 +22.5%
E4 可采纳边界	k = 0 → 6（c_min=3）	k≤3 全部最优；k=6 时 100% 次优但节点 −92%
E5 零代价反例	35% 零代价格 × 60 图	46.7% 次优，最大代价差 8，且出现“扩展更少”假象
E6 访问顺序	4 种顺序 × 60 棵随机树	最优优先 −88.6%；最差优先剪枝最多却最慢
E7 距离缓存	无缓存 / 按需缓存 / 预计算	H=4 求解 5.6 ms → 0.1 ms（约 56 倍）

表 10 七组对照实验一览（详细数据见前述各节）

三点结论。①最优性来自可采纳性，速度来自紧致性：曼哈顿在单位代价地图上是“恰好正确”的下界，换到加权地形虽仍可采纳却因过于保守而退化为 Dijkstra；用 c_min 缩放可恢复部分效率，越过下界立刻以次优为代价——E4 的 k=3 把这条理论边界精确地测了出来。②剪枝效率由访问顺序决定：同一份 α-β，最优优先省 93%，最差优先几乎不省，关键在于剪枝是否发生在浅层、能否整枝跳过。③博弈决策依赖深度与奇偶性：H=2 的贪心解与 H=4 的整局最优解不同，深层搜索能发现“弃子换先手”的策略，评价绝对值不可跨深度比较。

后续可探索：把加权 A* 与实时约束结合（Anytime Repairing A*）；在更大规模宝藏博弈上引入置换表与迭代加深；把静态排序升级为上一轮迭代的最佳走法（PV 排序）。

附录 代码结构与复现方式

```
qhf/                        # GitHub: https://github.com/zechun115/qhf.git
├── main.py                  # 四题入口，自动与题面预期输出比对
├── src/astar.py             # A*：4 种启发 + 3 种平局策略 + 加权 A*
├── src/game_tree.py         # Minimax / Alpha-Beta + 顺序重排 + 访问统计
├── src/treasure.py          # 题目四：距离矩阵 + 博弈状态 + 求解器
├── experiments/run_experiments.py   # E1~E7 数据采集（固定种子）
├── experiments/make_figures.py      # 报告插图
├── experiments/make_report.py       # 本 PDF 生成脚本
└── experiments/results/*.json       # 原始实验数据
```

复现：`python main.py` 输出四题结果并自检；`python experiments/run_experiments.py` 生成 results/*.json（约 10 秒）；再运行 `make_figures.py` 与 `make_report_short.py` 可重建全部插图与本报告。所有随机过程种子固定，任何机器上的数值与图表一致。